

K8S-01: Kubernetes Monitoring Fundamentals

Series: K8S — Kubernetes Monitoring | **Notebook:** 1 of 13 | **Created:** January 2026
| **Last Updated:** 09/18/2026

Introduction to Kubernetes Observability with Dynatrace

Kubernetes introduces unique observability challenges: ephemeral workloads, dynamic scaling, complex networking, and multi-layer abstractions. Dynatrace provides comprehensive Kubernetes monitoring through the DynaKube operator, which deploys and manages monitoring components automatically.

Table of Contents

1. [Kubernetes Observability Challenges](#)
 2. [Dynatrace Monitoring Architecture](#)
 3. [Entity Model for Kubernetes](#)
 4. [Data Sources and Signals](#)
 5. [Key Metrics and Dimensions](#)
 6. [Your First Kubernetes Queries](#)
 7. [Next Steps](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring enabled
Kubernetes Cluster	Any distribution (EKS, AKS, GKE, OpenShift, etc.)
Permissions	<code>ReadConfig</code> , <code>metrics.read</code> , <code>entities.read</code>
Knowledge	Basic Kubernetes concepts (pods, deployments, services)

1. Kubernetes Observability Challenges

Kubernetes environments present unique monitoring requirements:

Challenge	Description	Dynatrace Solution
Ephemeral Workloads	Pods come and go constantly	Entity relationships preserved across restarts
Dynamic Scaling	Replicas change based on load	Automatic discovery of new instances
Multi-Layer Stack	Infra → K8s → App complexity	Unified view from cluster to code
Distributed Services	Microservices across namespaces	End-to-end distributed tracing
Resource Constraints	CPU/memory limits and requests	Resource utilization vs. limits monitoring
Network Complexity	Service mesh, ingress, CNI	Network flow and latency analysis

Traditional vs. Cloud-Native Monitoring

Aspect	Traditional	Kubernetes
Identity	IP address, hostname	Labels, selectors, namespaces

Aspect	Traditional	Kubernetes
Lifecycle	Long-lived servers	Short-lived pods
Configuration	Static files	Dynamic ConfigMaps, Secrets
Networking	Fixed topology	Service discovery, DNS
Scaling	Manual or scheduled	HPA, VPA, KEDA

2. Dynatrace Monitoring Architecture

Dynatrace monitors Kubernetes through multiple components:

DynaKube Operator Components

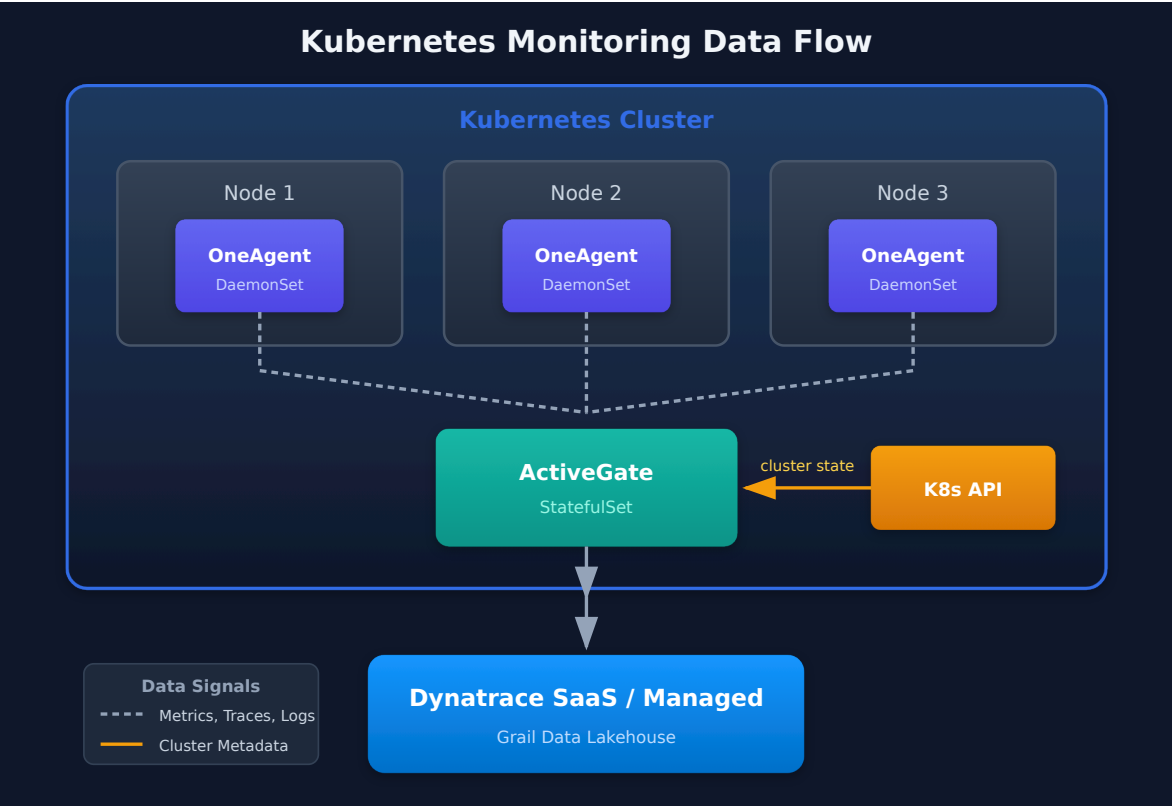
Component	Purpose	Deployment Mode
OneAgent	Full-stack monitoring (processes, code)	DaemonSet or application-only
ActiveGate	Routing, K8s API monitoring	StatefulSet in cluster
Kubernetes Monitoring	Cluster state, events	Via ActiveGate
Prometheus Integration	Custom metrics ingestion	Optional

Deployment Modes

Mode	Use Case	OneAgent	Code Modules
cloudNativeFullStack	Full visibility, K8s-native	Privileged DaemonSet	Injected via webhook
classicFullStack (legacy)	Traditional deployment — avoid for new deployments	Privileged DaemonSet	Loaded from host
applicationMonitoring	App-only, no infra	None	Injected via webhook

Mode	Use Case	OneAgent	Code Modules
hostMonitoring	Infra-only	DaemonSet	None

Data Flow



3. Entity Model for Kubernetes

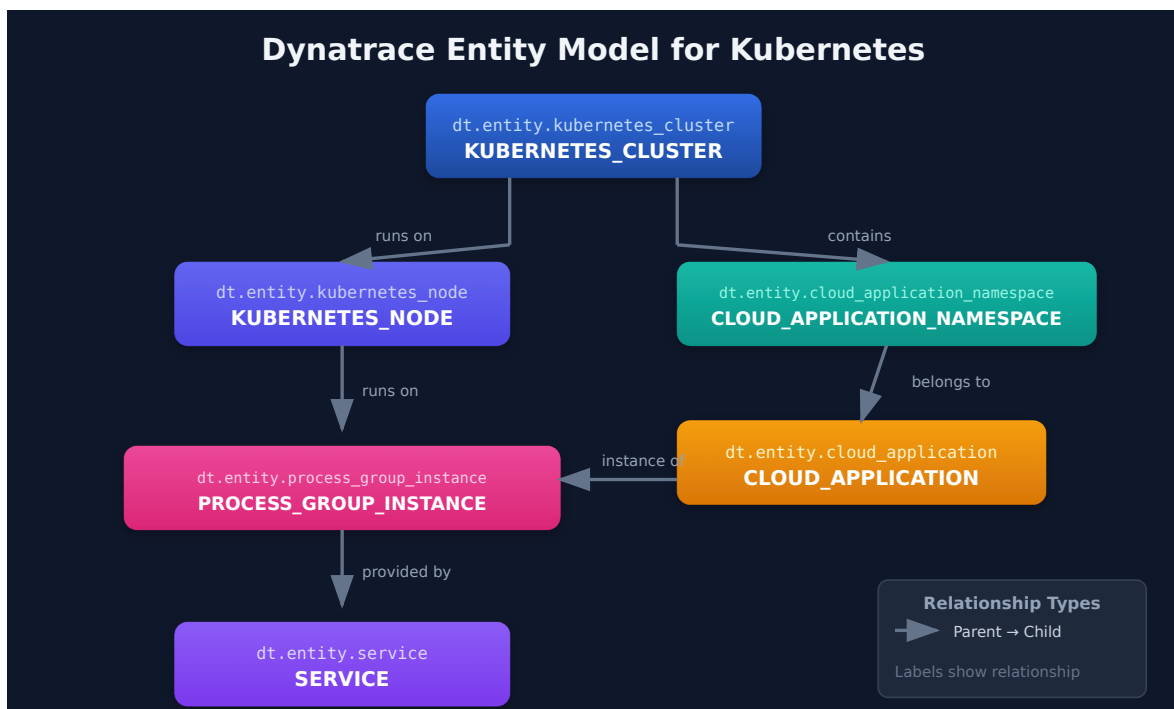
Dynatrace creates entities for each Kubernetes resource and maintains relationships between them.

Kubernetes Entity Types

Entity Type	Description	Key Attributes
KUBERNETES_CLUSTER	Cluster-level entity	Name, version, cloud provider
KUBERNETES_NODE	Worker nodes	CPU, memory, conditions

Entity Type	Description	Key Attributes
CLOUD_APPLICATION_NAMESPACE	Namespaces	Name, labels
CLOUD_APPLICATION	Deployments, StatefulSets	Replicas, strategy
PROCESS_GROUP_INSTANCE	Container processes	Image, resources
SERVICE	Detected services	Endpoints, technology

Entity Relationships



Entity Naming

Resource	Dynatrace Entity Name Pattern
Cluster	Cluster name from kubeconfig
Namespace	namespace - name
Deployment	deployment - name in namespace
Pod	pod - name (ephemeral, tied to PGI)
Service	Auto-detected from traffic patterns

4. Data Sources and Signals

Dynatrace collects multiple signal types from Kubernetes:

Metrics

Source	Metrics	Examples
Kubelet	Container resources	CPU, memory, network I/O
kube-state-metrics	Cluster state	Replica counts, conditions
cAdvisor	Container stats	Filesystem, limits
API Server	Control plane	Request latency, etcd
Custom	Prometheus endpoints	App-specific metrics

Logs

Log Type	Source	Use Case
Container logs	stdout/stderr	Application debugging
Kubernetes events	API server	Scheduling, scaling, errors
Audit logs	API server	Security, compliance
Node logs	kubelet, runtime	Infrastructure issues

Traces

Trace Source	Coverage
OneAgent auto-instrumentation	Supported languages/frameworks
OpenTelemetry	Custom instrumentation
Service mesh	Istio, Linkerd sidecars

5. Key Metrics and Dimensions

Container Resource Metrics

Metric	Description	Unit
<code>builtin:containers.cpu.usagePercent</code>	CPU usage vs. limit	Percent
<code>builtin:containers.memory.usagePercent</code>	Memory usage vs. limit	Percent
<code>builtin:containers.cpu.throttledTime</code>	Time CPU was throttled	Milliseconds
<code>builtin:containers.memory.workingSetBytes</code>	Working set memory	Bytes

Kubernetes Workload Metrics

Metric	Description	Unit
<code>builtin:kubernetes.workload.requests_cpu</code>	CPU requests	Millicores
<code>builtin:kubernetes.workload.requests_memory</code>	Memory requests	Bytes
<code>builtin:kubernetes.workload.limits_cpu</code>	CPU limits	Millicores
<code>builtin:kubernetes.workload.limits_memory</code>	Memory limits	Bytes

These are `builtin:` keys — the classic Metrics namespace, not Grail. They are correct for the classic Metrics API, Data Explorer and classic dashboards, but **DQL reads Grail only**: `metrics | filter startsWith(metric.key, "builtin:")` returns **zero rows over 7 days** on a live tenant (08/11/2026), while `startsWith(metric.key, "dt.kubernetes")` returns 27 keys in the same command. A `timeseries avg(builtin:kubernetes.workload.requests_cpu)` therefore charts nothing and reports no error.

For DQL, use the Grail equivalents — and note the **grain differs, not just the prefix**: Grail emits requests and limits at **container** grain

(`dt.kubernetes.container.requests_cpu` , `.requests_memory` , `.limits_cpu` , `.limits_memory`), summed by you across `k8s.workload.name` to reach a workload figure. There is no `dt.kubernetes.workload.requests_*` . Full Grail key list and the derivation pattern: **K8S-08 §2**.

Request and limit metrics changed what they count in ActiveGate 1.343 / 1.345 — they carry an *effective* pod value: from 1.343 init containers are accounted for, and from 1.345 pod-level `spec.resources` (Kubernetes 1.34+) and pod `overhead` too, which can move a limit *down*. Reserved figures can step at either upgrade with no workload change; usage metrics are unaffected. This matters whenever you compare a request/limit trend across an upgrade boundary — see K8S-08 §2 before drawing capacity conclusions from one.

Cluster Health Metrics

Metric	Description	Unit
<code>builtin:kubernetes.node.cpu_available</code>	Available CPU on nodes	Millicores
<code>builtin:kubernetes.node.memory_available</code>	Available memory on nodes	Bytes
<code>builtin:kubernetes.pods</code>	Pod count by state	Count

6. Your First Kubernetes Queries

Let's explore common queries for Kubernetes monitoring.

```
// Verify Kubernetes clusters are reporting (smartscape topology)
smartscapeNodes "K8S_CLUSTER"
| fields entity.name = name, tags
| sort entity.name asc

// Legacy alternative (deprecated for new content):
// fetch dt.entity.kubernetes_cluster
// | fields entity.name, tags
// | sort entity.name asc
```

```
// Count Kubernetes nodes
smartscapeNodes "K8S_NODE"
| summarize nodeCount = count()

// Legacy alternative:
// fetch dt.entity.kubernetes_node
// | summarize nodeCount = count()
```

```
// List Kubernetes namespaces
smartscapeNodes "K8S_NAMESPACE"
| fields entity.name = name, tags
| sort entity.name asc
| limit 50

// Legacy alternative:
// fetch dt.entity.cloud_application_namespace
// | fields entity.name, tags
// | sort entity.name asc
// | limit 50
```

```
// Container CPU usage - find highest consumers
timeseries avgCpuMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{dt.entity.container_group_instance}
| sort avgCpuMillicores desc
| limit 20
```

```
// Container memory usage approaching limits
timeseries avgMemBytes = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{dt.entity.container_group_instance}
| fieldsAdd avgMemBytesValue = arrayAvg(avgMemBytes)
| sort avgMemBytesValue desc
```

```

// Kubernetes events - recent warnings and errors
// Data object corrected 08/12/2026. Kubernetes events are NOT logs. This cell
scraped
// `fetch logs` for `log.source` containing "kubernetes" or for content
substrings like "BackOff" -
// no log.source matches, and kubelet event text is not in the log stream, so
it returned nothing
// while the cluster emitted 231,296 Kubernetes events in the same window.
// They arrive as `fetch events | filter event.provider ==
"KUBERNETES_EVENT"`, STRUCTURED:
//   dt.kubernetes.event.reason           Unhealthy · BackOff · Killing ·
FailedScheduling ·
//                                           FailedMount · BackoffLimitExceeded
· EvictionThresholdMet ...
//   dt.kubernetes.event.message           the human-readable text
//   dt.kubernetes.event.important         "true" marks the warning-class
events
//   dt.kubernetes.event.involved_object.kind / .name
//   dt.kubernetes.event.count / .first_seen / .last_seen
//   plus k8s.cluster.name · k8s.namespace.name · k8s.pod.name ·
k8s.workload.name · k8s.node.name
// NOTE: event.type is CUSTOM_INFO on every one of these – it is NOT
"Warning"; severity lives in
// dt.kubernetes.event.important. Enumerate reasons with:
//   fetch events, from:-24h | filter event.provider == "KUBERNETES_EVENT"
//   | summarize n = count(), by:{dt.kubernetes.event.reason} | sort n desc
fetch events, from:-1h
| filter event.provider == "KUBERNETES_EVENT"
| filter dt.kubernetes.event.important == "true"
| fields timestamp, k8s.cluster.name, k8s.namespace.name,
dt.kubernetes.event.reason, dt.kubernetes.event.message
| sort timestamp desc
| limit 25

```

```
// Pod restarts - find crashlooping workloads
// Data object corrected 08/12/2026. Kubernetes events are NOT logs. This cell
scraped
// `fetch logs` for `log.source` containing "kubernetes" or for content
substrings like "BackOff" –
// no log.source matches, and kubelet event text is not in the log stream, so
it returned nothing
// while the cluster emitted 231,296 Kubernetes events in the same window.
// They arrive as `fetch events | filter event.provider ==
"KUBERNETES_EVENT"`, STRUCTURED:
//   dt.kubernetes.event.reason           Unhealthy · BackOff · Killing ·
FailedScheduling ·
//                                           FailedMount · BackoffLimitExceeded
· EvictionThresholdMet ...
//   dt.kubernetes.event.message           the human-readable text
//   dt.kubernetes.event.important         "true" marks the warning-class
events
//   dt.kubernetes.event.involved_object.kind / .name
//   dt.kubernetes.event.count / .first_seen / .last_seen
//   plus k8s.cluster.name · k8s.namespace.name · k8s.pod.name ·
k8s.workload.name · k8s.node.name
// NOTE: event.type is CUSTOM_INFO on every one of these – it is NOT
"Warning"; severity lives in
// dt.kubernetes.event.important. Enumerate reasons with:
//   fetch events, from:-24h | filter event.provider == "KUBERNETES_EVENT"
//   | summarize n = count(), by:{dt.kubernetes.event.reason} | sort n desc
fetch events, from:-24h
| filter event.provider == "KUBERNETES_EVENT"
| filter in(dt.kubernetes.event.reason, {"BackOff", "BackoffLimitExceeded",
"Killing"})
| summarize events = count(), by:{k8s.namespace.name, k8s.pod.name,
dt.kubernetes.event.reason}
| sort events desc
| limit 25
```

7. Next Steps

Now that you understand Kubernetes monitoring fundamentals, proceed to:

Next Notebook	Topic
K8S-02: DynaKube Operator Deployment	Install and configure the operator
K8S-03: GitOps for DynaKube	Manage DynaKube with ArgoCD/Flux

Next Notebook	Topic
K8S-04: Cluster Health Monitoring	Deep-dive into cluster metrics

Summary

In this notebook, you learned:

- Kubernetes observability challenges and how Dynatrace addresses them
- Dynatrace monitoring architecture (OneAgent, ActiveGate, DynaKube)
- Entity model for Kubernetes resources
- Data sources: metrics, logs, and traces
- Key metrics for container and cluster monitoring
- Basic DQL queries for Kubernetes data using `smartscapeNodes` (modern) with `dt.entity.*` legacy alternatives shown as comments

References

- [Setup on Kubernetes \(DT docs\)](#) — top-level entry point for all K8s monitoring docs
- [How it works \(DT docs\)](#) — architecture, components, and data flow
- [Quickstart \(DT docs\)](#) — minimum viable deployment
- [Reference \(DT docs\)](#) — DynaKube parameters, feature flags, network, security, storage, workload mutation
- [Dynatrace Operator \(Dynatrace GitHub\)](#) — source, releases, and Helm chart
- [Effective pod resources \(DT docs\)](#) — how request/limit metrics represent the enforced pod value

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.